

Sentiment Classification Report

I. Introduction

In this project a train.csv dataset is provided for the training of the model created, which will be evaluated with an external dataset using unseen data. For the project some background research was done to try and understand the problem better and find the best possible approach. First, the text preprocessing is performed. Preprocessing involves duplicate data removal, parts of the text that provide no information is removed, normalization lowercasing the text, stopwords are removed and tokenization. The last step is lemmatization and stemming. This is where literature seems to have different approaches as it is mentioned that stemming is more important for sentiment analysis while other approaches use both methods or neither of them. Avinash et al 2019 does not use lemmatization or stemming while Noura A. et al 2024 uses both methods in their study. Regarding feature extraction, nearly every approach uses text transformations and word embeddings as sole input, the most famous ones being TF-IDF, Bag Of Words (BOW) and Word2Vec. For BOW and TF-IDF both unigrams and bigrams are suggested for the analysis (Razon A et al 2015) but also features like parts of speech tagging (Yixing C et al 2021), words showing negation like not, neither etc., or emojis, punctuation marks and slang are also evaluated (Wankhade M et al 2022). In the machine learning approach the classification algorithms widely used are Logistic regression (LR), Naïve Bayes (NB), Support Vector Machine (SVM), Random Forest (RF) and Decision Tree (Ravinder A et al 2019; Avinash M 2019; Wankhade M et al 2022).

II. Dataset

The training dataset consists of two columns, one with the text and the other with its corresponding sentiment. The labels are three types of emotion namely positive, negative and neutral and the text is small sentences probably taken through social media as many of them include hashtags and '@' characters. The dataset has 8000 entries, it has no specified origin e.g. reviews of customers, and the distribution of the labels is presented below.

A small imbalance is observed, positive label has the most entries, while negative the least one introducing some bias towards positive label classification. Specifically, the positive label is 38% of the dataset, the neutral 35% and the negative 27%.

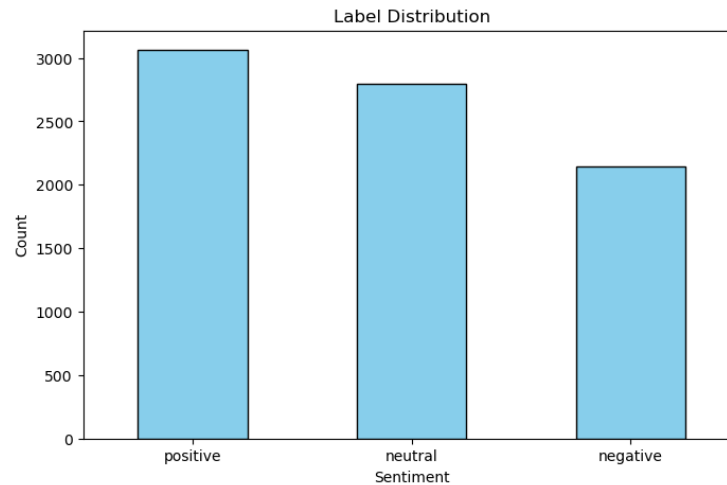


Figure 1: Sentiment distribution of train dataset

IV. Preprocessing

Firstly, the preprocessing of the text takes place. In my opinion it appeared to be the most important aspect of the procedure as it had a huge impact on the accuracy. Many different approaches and combinations were used and experimented on. Some of the approaches used are stopwords from NLTK, stemming using PorterStemmer, lemmatizing using WordNetLemmatizer, ReGex was used to remove any specific patterns, whitespaces, punctuations and numbers. Furthermore, the text was lowercased for normalization and extra white spaces removed to remove noise. For the tokenization WordTokenizer was used from NLTK.

The best accuracy, which was 5-6% above any other combination, was found using the following implementation. First of all, any numbers, punctuation, special characters were removed. Then the links were removed (http or www patterns) and the text was lowercased. Any additional whitespaces after the previous preprocessing were also removed. Tokenization was performed and stopwords were removed. And lastly only stemming was performed as it showed better accuracy when comparing to using lemmatizer. A barplot of the frequent words per sentiment label after preprocessing is shown below.

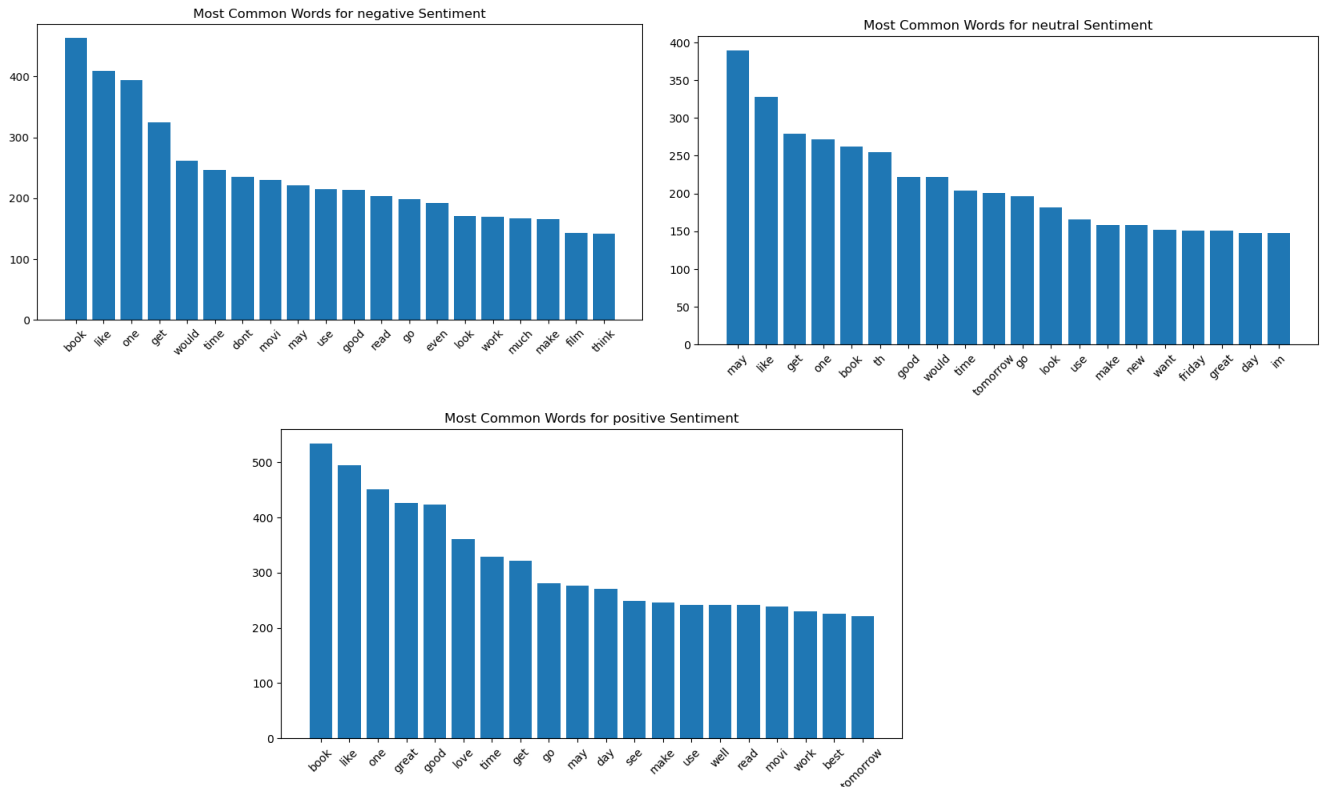


Figure 2: Most frequent words by sentiment

V. Feature Engineering

For the features used as input, the first thing was to perform the text representations. I used the BOW and TF-IDF approaches. I also tried word2vec but it had quite a bad accuracy, so I removed it from the features later.

Bag of Words (BoW) works by converting text into a numerical representation where each word in the vocabulary is treated as a separate feature. It creates a vector for each document where the values represent the count of words. TF-IDF, on the other hand, extends this approach by assigning a weight to each word based on its importance in the document relative to the entire corpus.

For TF-IDF and BOW I tried many parameter combinations starting from the range of continuous words considered. Namely for both approaches (1,1), (1,2), (1,3) were used. Then the minimum occurrence of a word to be taken into account is evaluated (min_df) and the max_df for filtering common words were used. Finally the maximum number of features (words or n-grams) to keep in the vectorized representation (max_features) was experimented on.

I also experimented with some other features either my intuition or from suggested ones in the bibliography. I tried some basic ones for each sentence like unique word ratio, average word length, text length and exclamation mark count. Also, I tried the pos_tags to extract the count of nouns, adjectives and adverbs. I also tried features like positive and negative word counts as well as negation words count, which were counted from lists of words I created on my own.

The features that I came down to after the whole process were the TF-IDF and the BOW but I also experimented with the other features. The rest of the features did not add any value to the accuracy, it mainly

remained the same or even lower, and I concluded that a more complicated model could perform worse on unseen data.

VI. Classification Algorithms

Based on the literature and my intuition I concluded to experiment on Logistic Regression, SVM, Random Forest and Naïve Bayes.

Logistic Regression models the probability of an outcome using a logistic function. It learns linear relationships between input features and the log-odds of the target variable. LR was selected because it is a baseline algorithm and has the ability for one vs rest classification. It is one of the most used, it is simple and quite interpretable. Also, it can handle high dimensional data such as text data.

SVM constructs a hyperplane in a high-dimensional space to separate data points of different classes. It maximizes the margin between the nearest points of the classes (support vectors) and the hyperplane. SVM was selected as considered one of the best performing algorithms among the machine learning ones and it was the state of the art before neural networks. It also works well with high dimensional data.

Random Forest was used instead of Decision Trees as it is an ensemble learning method that builds multiple decision trees using bootstrapped subsets of the data and random feature selection at each split. Each tree makes a prediction, and the final output is determined by majority voting. It is one of the high performing algorithms due to the ensemble effect, also it provides insights into feature importance to understand their contributions. It can also model nonlinear relationships.

Lastly, Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes that all features are conditionally independent given the class. It calculates the posterior probability for each class and assigns the label with the highest probability. Despite its simplicity, it performs remarkably well in text-based tasks, and it can handle sparse and high dimensional data. Furthermore, it is very computationally effective which makes it fast for large scale datasets, and a good option to get a quick evaluation of your features.

VII. Experiment

For the experiment a 90-10 train test split was used. In the training set Stratified 10-fold validation was used to address the imbalance, as stratified k=validation tries to keep the labels even. Additionally, the shuffle parameter was set to True to achieve randomness.

GridSearch and pipeline were used based on the accuracy metric, to fine tune the parameters of each algorithm and the different combinations of the text transformation parameters. Before searching through the parameters of text transformations, I tried a more focused GridSearch with searching through some parameters in the classifiers and the text transformations n_grams parameter manually tuned, to check the performance of the linguistic features engineered earlier. The features did not achieve any significant performance, even though I tried many combinations, so I decided to include only the text transformations as input features for less complicated models as the accuracy was the same.

Here the final experiment was executed, searching through parameters for both BOW, TF-IDF and the algorithms. Regarding the algorithms the following parameter combinations were used.

Classification Algorithm	Parameter #1	Parameter #2
LR	C: 0.1, 1, 10, 100	Solver: "lbfgs", "liblinear"
SVM	C: 0.1, 1, 10	Kernel: "linear", "rbf"
RF	N estimators: 50, 100, 200	Max depth: 10, 20, "None"
NB	Alpha: 0.1, 0.5, 1.0, 5.0, 10.0	-----

Table 1: Classification algorithms' parameters tuned

I tried to search through some of the basic parameters of each algorithm thinking that they play the more major role and will influence the most the accuracy. For SVM C is the regularization strength, where smaller values specify stronger regularization. Small values try to achieve a large margin of separation, allowing more misclassifications. Kernel specifies the kernel type used linear for linear separable data and rbf for nonlinear solutions. For LR C is once again a regularization term, the inverse of regularization strength, smaller values meaning stronger regularization. The Solver is the optimization algorithm lbfgs being suitable for multiclass classification with L2 regularization and efficient with large scale problems while liblinear being effective with smaller datasets, able to use both L1 and L2 regularization and uses one vs all for multiclass. For RF n estimators the number of trees in the forest, generally the more the better, and max depth

is the maximum depth of the tree, basically whether to restrict it or not. Lastly, NB has little customization basically alpha which is an additive smoothing parameter, high values smooth the probabilities more leading to a possible underfitting while for lower values we have overfitting.

VIII. Conclusion

In conclusion, after GridSearch was carried out the two best performing models were LR with TF-IDF and SVM with TF-IDF. The rest of the classifiers, all with TF-IDF as input, had an average accuracy below 0.56 so they are not taken into consideration. The best performing parameters were:

For TF-IDF: $n_grams = (1,2)$ so using both unigrams and bigrams and everything else default meaning max features has no limit, $min_df=1$, $max_df=1$.

For LR the regularization term $C = 10$, meaning the model is not very strict with liblinear solver.

TF-IDF with LR achieved a cross-validation accuracy of 0.5928 and a test accuracy 0.60.

For SVM the regularization term $C=1$ indicating medium strength, kernel = linear achieved cross validation accuracy 0.5886 and test accuracy 0.59875.

The F-1 scores for both models is about the same, each of them has a marginal benefit in some label.

	Negative	Neutral	Positive
F1-score LR	0.56	0.57	0.65
F1-Score SVM	0.55	0.57	0.66

Table 2: F-1 scores of best performing models

Here we see the Confusion matrixes for these two models:

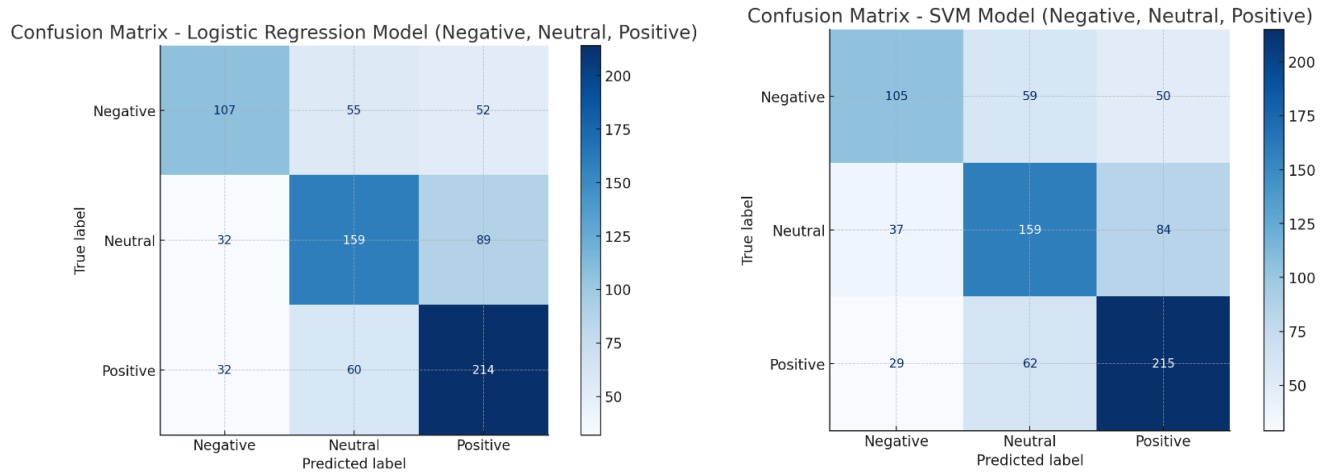


Figure 3: Confusion matrixes of the best performing models

The differences are very small but as the metric of choice for performance is accuracy, the Logistic Regression with TF-IDF is selected due to its marginal difference. The outcome of SVM being among the best models was expected, I did not expect LR to achieve so close and even better accuracy. I thought RF would perform but its performance was poor (average performance 0.55). NB had the least performance, but with faster results, mainly used as an indication for fast experiments having an average accuracy of 0.53. TF-IDF was expected to win over BOW with $n_grams = (1,2)$ being expected from literature.

REFERECES

- [1] Semary, N. A., Ahmed, W., Amin, K., Pławiak, P., & Hammad, M. (2024). Enhancing machine learning-based sentiment analysis through feature extraction techniques. *PLoS ONE*. Published February 14, 2024. <https://doi.org/10.1371/journal.pone.0294968>
- [2] Madasu, A., & E, S. (2019). A study of feature extraction techniques for sentiment analysis. *Computation and Language (cs.CL)*. arXiv:1906.01573. <https://doi.org/10.48550/arXiv.1906.01573>
- [3] Razon, A., & Barnden, J. (2015). A new approach to automated text readability classification based on concept indexing with integrated part-of-speech n-gram features. *Proceedings of the International Conference on Recent Advances in Natural Language Processing*, 521–528.
- [4] Wankhade, M., Rao, A. C. S., & Kulkarni, C. (2022). A survey on sentiment analysis methods, applications, and challenges. *Artificial Intelligence Review*, 55(9), 5731–5780. <https://doi.org/10.1007/s10462-022-10144-1>